

MERN NOTES

Absolutely! Let's start from scratch and walk through **core MERN concepts step-by-step**, like how you'd build a basic CRUD app. You'll get to **practice each part** — from `npm init` to React state — just like in the exam.

Step 1: Initialize Your Project

Let's set up the full MERN project structure.

Backend Setup

```
mkdir mern-practice
cd mern-practice
mkdir backend
cd backend
npm init -y
npm install express mongoose cors
```

Create `server.js` inside `backend/`

```
// backend/server.js
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');

const app = express();
app.use(express.json());
app.use(cors());

mongoose.connect('your_mongodb_url_here', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});

// Define a sample schema
const Note = mongoose.model('Note', {
  title: String,
  content: String,
});

// API routes
app.get('/api/notes', async (req, res) => {
  const notes = await Note.find();
  res.json(notes);
});

app.post('/api/notes', async (req, res) => {
  const note = new Note(req.body);
  await note.save();
  res.json(note);
});
```

```
app.listen(5000, () => console.log('Backend running on  
http://localhost:5000'));
```

✅ Let me know once you're done with this step and the server is running. Then we'll move on to creating the **frontend with React**.

Perfect! 🎉 Now let's set up the **frontend with React**.

🔧 Step 2: Frontend Setup (React)

In your main mern-practice/ folder:

```
cd ..  
npx create-react-app frontend  
cd frontend  
npm install axios
```

Now clean up the default files a bit:

```
rm -rf src/App.test.js src/logo.svg src/setupTests.js  
src/reportWebVitals.js
```

🧠 Step 3: Create a Simple Notes App (React)

We'll fetch and display notes from your backend and allow users to add new ones.

✅ Update **src/App.js**:

```
import React, { useEffect, useState } from 'react';  
import axios from 'axios';  
  
function App() {  
  const [notes, setNotes] = useState([]);  
  const [form, setForm] = useState({ title: '', content: '' });  
  
  useEffect(() => {  
    axios.get('http://localhost:5000/api/notes')  
      .then(res => setNotes(res.data))  
      .catch(err => console.error(err));  
  }, []);  
  
  const handleChange = (e) => {  
    setForm({ ...form, [e.target.name]: e.target.value });  
  };  
  
  const handleSubmit = async (e) => {  
    e.preventDefault();  
    const res = await axios.post('http://localhost:5000/api/notes', form);  
    setNotes([...notes, res.data]);  
    setForm({ title: '', content: '' });  
  };  
};
```

```

return (
  <div style={{ padding: '2rem' }}>
    <h1> 📝 Notes App</h1>
    <form onSubmit={handleSubmit}>
      <input
        name="title"
        placeholder="Title"
        value={form.title}
        onChange={handleChange}
        required
      />
      <br />
      <textarea
        name="content"
        placeholder="Content"
        value={form.content}
        onChange={handleChange}
        required
      />
      <br />
      <button type="submit">Add Note</button>
    </form>

    <h2>All Notes:</h2>
    <ul>
      {notes.map(note => (
        <li key={note._id}>
          <strong>{note.title}</strong>: {note.content}
        </li>
      ))}
    </ul>
  </div>
);
}

export default App;

```

🔄 Run your app:

npm start

Make sure your **backend is still running** on port 5000 and frontend is on 3000.

✅ You Just Practiced:

- npm, modules, and basic backend with Express
 - MongoDB + Mongoose connection
 - React app, props/state/event handlers
 - Axios to call backend API
 - Full MERN integration 🎯
-

No worries! You're using **PowerShell**, so the `rm -rf` command (which works in Bash/Linux/macOS) won't work there.

In PowerShell, use this instead to delete the default files:

```
Remove-Item -Path src\App.test.js, src\logo.svg, src\setupTests.js,
src\reportWebVitals.js
```

Or just delete them manually in File Explorer from this folder:

```
mern-practice/frontend/src/
```

✅ **Once deleted, update your `App.js` as we discussed earlier.**

Here's a quick reminder — replace everything in `frontend/src/App.js` with:

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

function App() {
  const [notes, setNotes] = useState([]);
  const [form, setForm] = useState({ title: '', content: '' });

  useEffect(() => {
    axios.get('http://localhost:5000/api/notes')
      .then(res => setNotes(res.data))
      .catch(err => console.error(err));
  }, []);

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const res = await axios.post('http://localhost:5000/api/notes', form);
    setNotes([...notes, res.data]);
    setForm({ title: '', content: '' });
  };

  return (
    <div style={{ padding: '2rem' }}>
      <h1> 📝 Notes App</h1>
      <form onSubmit={handleSubmit}>
        <input
          name="title"
          placeholder="Title"
          value={form.title}
          onChange={handleChange}
          required
        />
        <br />
        <textarea
          name="content"
          placeholder="Content"
          value={form.content}
        />
      </form>
    </div>
  );
}
```

```

        onChange={handleChange}
        required
      />
      <br />
      <button type="submit">Add Note</button>
    </form>

    <h2>All Notes:</h2>
    <ul>
      {notes.map(note => (
        <li key={note._id}>
          <strong>{note.title}</strong>: {note.content}
        </li>
      ))}
    </ul>
  </div>
);
}

export default App;

```

Once that's done, go ahead and run:

```
npm start
```

So your `index.js` should look like this:

```

import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
// import reportWebVitals from './reportWebVitals';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

// reportWebVitals();

```

Great initiative — understanding the full **MERN flow** will help you ace your exam confidently. Here's a **complete explanation** of how this basic Notes app works, both **backend and frontend**, and where/how you would update or add new code.



MERN Project Overview: Notes App

```

mern-practice/
├── backend/
│   └── server.js          <-- Express server + routes + MongoDB connection

```

```
└─ frontend/
  └─ src/
    └─ App.js      <-- Main React component (form + note list)
    └─ index.js    <-- Entry point of React app
```

Backend (Express + MongoDB with Mongoose)

File: backend/server.js

a. Imports & Setup

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
```

```
const app = express();
app.use(express.json());
app.use(cors());
```

- `express.json()` parses incoming JSON requests (needed for POST)
- `cors()` allows frontend (localhost:3000) to access backend (localhost:5000)

b. MongoDB Connection

```
mongoose.connect('mongodb+srv://<username>:<password>@cluster.mongodb.net/dbname');
```

- Connects to a MongoDB Atlas cluster
- If this fails → no DB operations will work

c. Schema & Model

```
const Note = mongoose.model('Note', {
  title: String,
  content: String,
});
```

- A **Note** document has `title` and `content` (both strings)
- Mongoose auto-creates a `notes` collection

d. Routes (API Endpoints)

```
app.get('/api/notes', async (req, res) => {
  const notes = await Note.find();
  res.json(notes);
});
```

- Fetches all notes from the DB

```
app.post('/api/notes', async (req, res) => {
  const note = new Note(req.body);
```

```
    await note.save();
    res.json(note);
  });
```

- Creates a new note in DB using request body (`req.body`)

e. Start Server

```
app.listen(5000, () => console.log('Backend running on
http://localhost:5000'));
```

- Starts the server on port 5000
-

2 Frontend (React)

File: `frontend/src/App.js`

a. React Imports

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';
```

- `useState` for managing state (notes + form)
- `useEffect` to run logic when the component first loads
- `axios` to send HTTP requests

b. Local State

```
const [notes, setNotes] = useState([]);
const [form, setForm] = useState({ title: '', content: '' });
```

- `notes`: array of notes from the backend
- `form`: holds form input values

c. Fetch Notes on Load

```
useEffect(() => {
  axios.get('http://localhost:5000/api/notes')
    .then(res => setNotes(res.data))
    .catch(err => console.error(err));
}, []);
```

- Runs **once** after the component mounts
- Sends `GET` request to fetch notes from the backend

d. Handle Form Input

```
const handleChange = (e) => {
  setForm({ ...form, [e.target.name]: e.target.value });
};
```

- Updates the `form` state when user types in inputs

e. Handle Submit (POST Request)

```
const handleSubmit = async (e) => {
  e.preventDefault();
  const res = await axios.post('http://localhost:5000/api/notes', form);
  setNotes([...notes, res.data]);
  setForm({ title: '', content: '' });
};
```

- Prevents page reload (`e.preventDefault()`)
- Sends form data to backend using `POST`
- Updates frontend state with new note

f. Render UI

```
<form onSubmit={handleSubmit}> ... </form>

<ul>
  {notes.map(note => (
    <li key={note._id}>
      <strong>{note.title}</strong>: {note.content}
    </li>
  ))}
</ul>
```

III Where to Insert or Update Code

Goal	File	What to Update
Add new data field (e.g., tag)	backend/server.js	Update Mongoose schema + frontend form/state
Add delete note feature	backend/server.js + App.js	Add a DELETE route in Express + delete button in React
Add update note feature	backend/server.js + App.js	Add PUT route + form/edit logic in React
Filter notes by title	backend/server.js	Add query param handling in <code>GET /api/notes</code>
Auth (Login/Signup)	backend: add <code>/api/auth</code> routes + frontend: add login/signup components	
EJS page for summary	backend/views/ + configure EJS view engine in <code>server.js</code>	

IV Summary

This app is a **full working MERN app**:

- Backend listens for API requests and talks to MongoDB
- Frontend talks to backend using Axios and displays results
- You control everything via `state`, `props`, and `fetch/axios`

That's a great idea — let's break it down into a **generic CRUD + feature extension workflow** that you can apply to **any MERN stack app** during your exam or projects.

✅ Generic MERN Stack Feature Extension Cheat Sheet

This template will help you **add or modify features** in a structured way — works for Notes app, Blog, Tasks, Users, etc.

📦 Add New Data Field (e.g., `tag`)

Step	Where	What to Do
Define in Schema	<code>backend/models/Model.js</code> or <code>server.js</code>	Add new field to Mongoose schema
Update API Handling	<code>POST</code> , <code>PUT</code> routes	Accept & save new field
Update Form	<code>App.js</code> or component	Add input in form, update <code>useState</code>
Update Display	<code>App.js</code> or component	Show new field in list/card

🗑️ Add Delete Feature

Step	Where	What to Do
Backend Route	<code>server.js</code> or <code>routes/notes.js</code>	Add <code>DELETE /api/items/:id</code> route
Frontend Event	<code>App.js</code>	Add Delete button and handler function
API Call	<code>axios.delete('/api/items/\${id}')</code>	Remove item and update state

🔄 Add Update/Edit Feature

Step	Where	What to Do
Backend Route	<code>PUT /api/items/:id</code>	Accept and update fields

Step	Where	What to Do
Frontend Form	Allow editing (prefill inputs)	Create edit button + prefill form
API Call	<code>axios.put('/api/items/\${id}', updatedData)</code>	On submit, send updated data
Update UI	Replace old item in state	<code>setItems(prev => prev.map(...))</code>

🔍 Filter/Search (Query Params)

Step	Where	What to Do
Backend Route	<code>GET /api/items?title=abc</code>	Use <code>req.query</code> to filter results
Mongo Query	<code>.find({ title: new RegExp(query, 'i') })</code>	Search ignoring case
Frontend	Add search box, update <code>useEffect()</code>	Call API with query param on change

🔑 Auth: Login / Signup

Step	Where	What to Do
Backend Routes	<code>/api/auth/register,</code> <code>/api/auth/login</code>	Add user schema + password hashing (bcrypt)
JWT Tokens	Sign JWT on login, send to frontend	Store token in <code>localStorage</code>
Frontend Forms	Login + Signup components	Send <code>POST</code> requests to backend
Auth Guard	Check token to protect routes	Add <code>PrivateRoute</code> or conditionally render pages

📄 Render with EJS (Server-side view)

Step	Where	What to Do
Install + Setup	<code>npm install ejs</code>	<code>app.set('view engine', 'ejs')</code>
Create EJS File	<code>views/report.ejs</code>	Use <code><%= data %></code> to display
Route	<code>GET /report</code>	<code>res.render('report', { data: value })</code>

✅ Reusable Code Templates

Mongoose Schema (Generic)

```
const mongoose = require('mongoose');
```

```
const ItemSchema = new mongoose.Schema({
  title: String,
  content: String,
  tag: String,
});
module.exports = mongoose.model('Item', ItemSchema);
```

CRUD Routes

```
app.get('/api/items', async (req, res) => {
  const { title } = req.query;
  const items = title
    ? await Item.find({ title: new RegExp(title, 'i') })
    : await Item.find();
  res.json(items);
});

app.post('/api/items', async (req, res) => {
  const item = new Item(req.body);
  await item.save();
  res.json(item);
});

app.delete('/api/items/:id', async (req, res) => {
  await Item.findByIdAndDelete(req.params.id);
  res.sendStatus(204);
});

app.put('/api/items/:id', async (req, res) => {
  const updated = await Item.findByIdAndUpdate(req.params.id, req.body,
  { new: true });
  res.json(updated);
});
```

React Axios Usage

```
// Create
axios.post('/api/items', data);

// Read
axios.get('/api/items?title=filter');

// Update
axios.put(`/api/items/${id}`, updatedData);

// Delete
axios.delete(`/api/items/${id}`);
```

Summary: Generic Feature Development Flow

1. **Decide feature (e.g. edit/delete/filter/tag/auth)**
2. **Update schema/model** if needed
3. **Update or add backend routes**
4. **Connect frontend forms/buttons to API**
5. **Update state/UI accordingly**

6. Test everything end-to-end

Perfect — let's build out all the core MERN features **step-by-step with full code and explanations**.

We'll enhance your existing **Notes App** to include:

✓ Features to Cover (One by One)

1. Add New Field (e.g. `tag`)
 2. Delete Note
 3. Update/Edit Note
 4. Filter/Search Notes by Title
 5. Login / Signup Authentication with JWT
 6. Render EJS Page for Note Summary
-

We'll do it in chunks. Let's begin with **Feature 1: Add New Field (Tag)**.

□ Add New Field (`tag`) to Note

✓ Backend: `server.js`

Update the schema:

```
const Note = mongoose.model('Note', {
  title: String,
  content: String,
  tag: String, // NEW FIELD
});
```

No need to change anything in GET/POST routes — Mongoose will auto-handle the extra field.

✓ Frontend: `App.js`

Update state:

```
const [form, setForm] = useState({ title: '', content: '', tag: '' });
```

Update form JSX:

```
<input
  name="tag"
  placeholder="Tag"
  value={form.tag}
  onChange={handleChange}
/>
```

Update UI rendering:

```
<ul>
  {notes.map(note => (
    <li key={note._id}>
      <strong>{note.title}</strong> ({note.tag}): {note.content}
    </li>
  ))}
</ul>
```

✓ Result

- Users can now add a **tag field** when creating notes
 - Notes display `title (tag): content`
-

Awesome! Let's move to **Feature 2: Delete Note** with full backend and frontend code.

2 Delete Note

✓ Backend: `server.js`

Add this new route at the bottom (after POST):

```
app.delete('/api/notes/:id', async (req, res) => {
  try {
    await Note.findByIdAndDelete(req.params.id);
    res.sendStatus(204); // No content
  } catch (err) {
    res.status(500).json({ error: 'Delete failed' });
  }
});
```

✓ Frontend: `App.js`

1. Add this function below `handleSubmit`:

```
const handleDelete = async (id) => {
```

```
    await axios.delete(`http://localhost:5000/api/notes/${id}`);
    setNotes(notes.filter(note => note._id !== id));
  };
};
```

2. Add delete button inside `map()` loop:

```
{notes.map(note => (
  <li key={note._id}>
    <strong>{note.title}</strong> ({note.tag}): {note.content}
    <button onClick={() => handleDelete(note._id)}>Delete</button>
  </li>
))}
```

✓ Result

- Each note has a **Delete** button
 - Clicking it sends a `DELETE` request to backend
 - React state is updated immediately (note removed)
-

Perfect! Let's now implement **Feature 3: Update/Edit Note** — this will let users update existing notes.

We'll need to:

- Prefill the form when the user clicks **Edit**
 - Change the submit behavior based on whether it's **adding** or **editing**
 - Update the backend with a `PUT` route
-

📦 Update/Edit Note

✓ Backend: `server.js`

Add this new `PUT` route:

```
app.put('/api/notes/:id', async (req, res) => {
  try {
    const updatedNote = await Note.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );
    res.json(updatedNote);
  } catch (err) {
    res.status(500).json({ error: 'Update failed' });
  }
});
```

```
});
```

✓ Frontend: App.js

We need to manage edit mode using state:

1. Add a new state:

```
const [editingNoteId, setEditingNoteId] = useState(null);
```

2. Update `handleSubmit()` to check if we're editing:

```
const handleSubmit = async (e) => {
  e.preventDefault();

  if (editingNoteId) {
    // Editing
    const res = await
    axios.put(`http://localhost:5000/api/notes/${editingNoteId}`, form);
    setNotes(notes.map(note => note._id === editingNoteId ? res.data :
    note));
    setEditingNoteId(null);
  } else {
    // Creating
    const res = await axios.post('http://localhost:5000/api/notes', form);
    setNotes([...notes, res.data]);
  }

  setForm({ title: '', content: '', tag: '' });
};
```

3. Add `handleEdit()` to fill the form:

```
const handleEdit = (note) => {
  setForm({ title: note.title, content: note.content, tag: note.tag });
  setEditingNoteId(note._id);
};
```

4. Update note rendering:

```
{notes.map(note => (
  <li key={note._id}>
    <strong>{note.title}</strong> ({note.tag}): {note.content}
    <button onClick={() => handleEdit(note)}>Edit</button>
    <button onClick={() => handleDelete(note._id)}>Delete</button>
  </li>
))}
```

5. Optional: Change form button text:

```
<button type="submit">
  {editingNoteId ? 'Update Note' : 'Add Note'}
</button>
```

✓ Result

- Clicking **Edit** fills the form with that note's data
- Button becomes "Update Note"
- On submit, it sends a `PUT` request and updates the list

Awesome! Let's implement **Feature 4: Filter/Search Notes by Title** — this will allow users to search for notes by their title in real-time.

4. Filter/Search Notes by Title

✓ Backend: `server.js`

Update your **GET route** to handle a `title` query parameter:

```
app.get('/api/notes', async (req, res) => {
  const { title } = req.query;

  let query = {};
  if (title) {
    // Case-insensitive search
    query.title = { $regex: title, $options: 'i' };
  }

  const notes = await Note.find(query);
  res.json(notes);
});
```

✓ Frontend: `App.js`

1. Add search state:

```
const [search, setSearch] = useState('');
```

2. Add search input above the form:

```
<input
  type="text"
  placeholder="Search by title"
  value={search}
  onChange={(e) => setSearch(e.target.value)}
  style={{ marginBottom: '1rem' }}
/>
```

3. Update `useEffect` to refetch notes when search changes:

```
useEffect(() => {
```



```
const fetchNotes = async () => {
  const res = await
  axios.get(`http://localhost:5000/api/notes?title=${search}`);
  setNotes(res.data);
};

fetchNotes();
}, [search]);
```

✓ Result

- As users type in the search bar, the note list updates live
 - Title matching is case-insensitive
 - Full-text search on the `title` field
-

Awesome — now we're getting into **Feature 5: Login/Signup Authentication with JWT**. This is super important for any MERN project and commonly tested in exams.

5 Login / Signup with JWT

We'll build a simple auth system with:

- Signup route to register a user
 - Login route to authenticate and return a JWT
 - Protecting API routes using middleware
-

✓ Step-by-Step Implementation

Backend Folder Structure

Update your `backend/` folder like this:

```
backend/
├── server.js
├── models/
│   └── User.js
├── routes/
│   └── auth.js
├── middleware/
│   └── authMiddleware.js
```

❏ Install Dependencies

In backend/:

```
npm install bcryptjs jsonwebtoken
```

❏ models/User.js

```
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true },
});

module.exports = mongoose.model('User', UserSchema);
```

❏ routes/auth.js

```
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const router = express.Router();
const JWT_SECRET = 'secret123'; // Use env var in real apps

// Register
router.post('/register', async (req, res) => {
  const { username, password } = req.body;

  const hashedPassword = await bcrypt.hash(password, 10);
  try {
    const user = await User.create({ username, password: hashedPassword });
    res.json({ message: 'User created', user });
  } catch (err) {
    res.status(400).json({ error: 'Username may already exist' });
  }
});

// Login
router.post('/login', async (req, res) => {
  const { username, password } = req.body;

  const user = await User.findOne({ username });
  if (!user) return res.status(400).json({ error: 'User not found' });

  const valid = await bcrypt.compare(password, user.password);
  if (!valid) return res.status(401).json({ error: 'Incorrect password' });

  const token = jwt.sign({ id: user._id }, JWT_SECRET, { expiresIn:
'1h' });
  res.json({ token });
});
```

```
module.exports = router;
```

4 middleware/authMiddleware.js

```
const jwt = require('jsonwebtoken');
const JWT_SECRET = 'secret123';

function authMiddleware(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer '))
    return res.status(401).json({ error: 'No token provided' });

  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    res.status(403).json({ error: 'Invalid token' });
  }
}

module.exports = authMiddleware;
```

5 Update server.js

```
const authRoutes = require('./routes/auth');
const authMiddleware = require('./middleware/authMiddleware');

app.use('/api/auth', authRoutes);

// Example: Protect this route
app.get('/api/secure-notes', authMiddleware, async (req, res) => {
  const notes = await Note.find();
  res.json(notes);
});
```

✓ Frontend (React)

You can add:

- Login.jsx, Register.jsx components
- Store token in localStorage
- Attach token to requests using
 axios.defaults.headers.common['Authorization']

Let me know and I'll give you full frontend auth flow next (with login form, token storage, protected routes, etc).

✅ Auth system now added in backend!

Perfect! Let's complete **both remaining features**:

1. **Frontend Integration for Login/Signup with JWT**
2. **EJS Page Rendering from the Backend**

5B: Frontend Login/Signup with JWT

✅ 1. Add Login and Register Forms in React

Create two new components in `frontend/src`:

`Login.jsx`

```
import React, { useState } from 'react';
import axios from 'axios';

function Login({ onLogin }) {
  const [form, setForm] = useState({ username: '', password: '' });

  const handleChange = e => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleSubmit = async e => {
    e.preventDefault();
    try {
      const res = await axios.post('http://localhost:5000/api/auth/login',
form);
      localStorage.setItem('token', res.data.token);
      onLogin();
    } catch (err) {
      alert('Login failed');
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <h2>Login</h2>
      <input name="username" placeholder="Username" onChange={handleChange} />
      <input name="password" type="password" placeholder="Password"
onChange={handleChange} />
      <button type="submit">Login</button>
    </form>
  );
}
```

```
export default Login;
```



Register.jsx

```
import React, { useState } from 'react';
import axios from 'axios';

function Register() {
  const [form, setForm] = useState({ username: '', password: '' });

  const handleChange = e => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleSubmit = async e => {
    e.preventDefault();
    try {
      await axios.post('http://localhost:5000/api/auth/register', form);
      alert('User registered! Now login.');
```

```
    } catch (err) {
      alert('Registration failed');
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <h2>Register</h2>
      <input name="username" placeholder="Username" onChange={handleChange}
    />
      <input name="password" type="password" placeholder="Password"
    onChange={handleChange} />
      <button type="submit">Register</button>
    </form>
  );
}

export default Register;
```



2. Update App.js to Use Auth Token

- Check login using `localStorage.getItem('token')`
- Pass Authorization header in axios

```
useEffect(() => {
  const fetchNotes = async () => {
    const token = localStorage.getItem('token');
    const res = await axios.get('http://localhost:5000/api/secure-notes', {
      headers: { Authorization: `Bearer ${token}` }
    });
    setNotes(res.data);
  };

  fetchNotes();
}, []);
```

✓ 3. Setup Routing (Optional)

You can switch between Register, Login, and Notes app using a simple `useState` or `react-router-dom`.

✓ Now your frontend is secured with JWT login & signup!
Let's now add...



EJS Page Rendering (Server-Side HTML)

✓ 1. Install EJS in Backend

```
npm install ejs
```

✓ 2. Update `server.js`

```
app.set('view engine', 'ejs');  
app.set('views', './views');
```

✓ 3. Create `views/report.ejs`

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Note Summary</title>  
</head>  
<body>  
  <h1>Note Report</h1>  
  <ul>  
    <% notes.forEach(note => { %>  
      <li><strong><%= note.title %></strong> (<%= note.tag %>): <%=  
note.content %></li>  
      <% }) %>  
    </ul>  
</body>  
</html>
```

✓ 4. Add Route to Render Report

```
app.get('/report', async (req, res) => {  
  const notes = await Note.find();  
  res.render('report', { notes });  
});
```

```
});
```

✅ Result

- Visiting `http://localhost:5000/report` renders a clean **HTML page** with note summary
 - Great for print/export-style reports
-

You Now Have:

- ✅ Full CRUD (Create, Read, Update, Delete)
 - ✅ Filters (Search)
 - ✅ Authentication (Login + Signup + JWT protection)
 - ✅ Server-side rendering with EJS
-

Great question! Understanding when and **why you need `body-parser` or `express.json()`** is super important for working with APIs in Express.

✅ When Do You Need `body-parser` or `express.json()`?

Short Answer:

You need it **whenever you want to read JSON data from the request body** in Express routes like `POST`, `PUT`, or `PATCH`.

Background

In Express 4.16+, `express.json()` is built-in and **replaces** the need to install `body-parser` separately for JSON.

So in modern apps:

```
app.use(express.json());
```

= ✅ You're good to handle JSON.

Why Do We Need It?

When a client sends a request with a JSON body like:

```
POST /api/notes
Content-Type: application/json

{
  "title": "Hello",
  "content": "World"
}
```

This data lives in `req.body`, but **Express doesn't parse it by default**. That's why you need:

```
app.use(express.json());
```

Without this line, `req.body` will be `undefined` for JSON payloads.

When To Use It (Common Scenarios)

Scenario	Required?	Parser Needed
POST a form using JSON	✓ Yes	<code>express.json()</code>
PUT or PATCH JSON data	✓ Yes	<code>express.json()</code>
GET with query params	✗ No	Not needed (<code>req.query</code>)
POST a regular HTML form	✓ Yes	<code>express.urlencoded()</code>
Send file/form-data (e.g. image upload)	✓ Yes	<code>multer</code> (not <code>body-parser</code>)

Modern Setup (Recommended)

In `server.js`:

```
const express = require('express');
const cors = require('cors');

const app = express();
app.use(express.json()); // for parsing application/json
app.use(cors());
```

If you're also accepting HTML form submissions (`application/x-www-form-urlencoded`):

```
app.use(express.urlencoded({ extended: true }));
```

When You Don't Need It

- GET requests (you use `req.query`)
 - DELETE requests that don't have a body
 - Static file serving
-



Summary

Use case	Middleware needed
JSON body (API POST/PUT)	<code>express.json()</code>
HTML form submission	<code>express.urlencoded()</code>
File uploads	<code>multer</code> or similar

Perfect! Here's your **Web Engineering Practice Pack**  with:

-  **Common Bugs** (with explanations + fixes)
 -  **Feature Requests** (with possible code solutions)
-



BUGS & FIXES (Frontend + Backend)



1. React not fetching data

```
useEffect(() => {  
  fetch('http://localhost:5000/api/notez') // ❌ Typo in URL  
    .then(res => res.json())  
    .then(data => setNotes(data));  
}, []);
```



Fix:

```
fetch('http://localhost:5000/api/notes')
```



2. CORS Error in Frontend Console

Backend:

```
const express = require('express');  
const app = express();  
app.use(express.json()); // ❌ Missing CORS
```



Fix:

```
const cors = require('cors');
app.use(cors());
```

3. Form Input Doesn't Update (React)

```
<input value={form.title} onChange={handleChange} />
const [form, setForm] = useState(); //  undefined
```

Fix:

```
const [form, setForm] = useState({ title: '', content: '' });
```

4. MongoDB Connection Fails

```
mongoose.connect('mongodb://localhost:27017'); //  Not specifying DB name
```

Fix:

```
mongoose.connect('mongodb://localhost:27017/notes_app');
```

5. POST works, but nothing is saved

Cause: Async/await missing

```
app.post('/api/notes', (req, res) => {
  const note = new Note(req.body);
  note.save(); //  Not awaited
  res.send(note);
});
```

Fix:

```
app.post('/api/notes', async (req, res) => {
  const note = new Note(req.body);
  await note.save();
  res.json(note);
});
```

6. useEffect Runs Forever

```
useEffect(() => {
  fetchNotes();
}); //  Missing dependency array
```

Fix:

```
useEffect(() => {  
  fetchNotes();  
}, []); // ✅ Runs once
```

+ FEATURE TASKS & SOLUTIONS

💡 Feature 1: Add New Field (e.g., tag)

Backend – Add to Schema:

```
const Note = mongoose.model('Note', {  
  title: String,  
  content: String,  
  tag: String // ✅ new field  
});
```

Frontend – Update Form:

```
<input name="tag" value={form.tag} onChange={handleChange} />
```

Frontend – Update State:

```
const [form, setForm] = useState({ title: '', content: '', tag: '' });
```

💡 Feature 2: Add DELETE Functionality

Backend:

```
app.delete('/api/notes/:id', async (req, res) => {  
  await Note.findByIdAndDelete(req.params.id);  
  res.sendStatus(204);  
});
```

Frontend:

```
<button onClick={() => handleDelete(note._id)}>Delete</button>  
  
const handleDelete = async (id) => {  
  await axios.delete(`http://localhost:5000/api/notes/${id}`);  
  setNotes(notes.filter(n => n._id !== id));  
};
```

💡 Feature 3: Edit/Update Notes

Backend:

```
app.put('/api/notes/:id', async (req, res) => {
```

```
    const updated = await Note.findByIdAndUpdate(req.params.id, req.body,
{ new: true });
    res.json(updated);
  });
```

Frontend:

```
const handleEdit = (note) => {
  setForm({ title: note.title, content: note.content, tag: note.tag });
  setEditingNoteId(note._id);
};

const handleSubmit = async () => {
  if (editingNoteId) {
    const res = await axios.put(`/api/notes/${editingNoteId}`, form);
    setNotes(notes.map(n => n._id === editingNoteId ? res.data : n));
    setEditingNoteId(null);
  }
};
```

💡 Feature 4: Search Notes by Title

Frontend – Search Input:

```
<input
  type="text"
  value={search}
  onChange={e => setSearch(e.target.value)}
/>
```

Backend:

```
app.get('/api/notes', async (req, res) => {
  const { title } = req.query;
  const query = title ? { title: { $regex: title, $options: 'i' } } : {};
  const notes = await Note.find(query);
  res.json(notes);
});
```

💡 Feature 5: Login + JWT Auth

Backend – auth.js:

```
const token = jwt.sign({ id: user._id }, 'secret123');
res.json({ token });
```

Middleware:

```
const decoded = jwt.verify(token, 'secret123');
req.user = decoded;
```

Frontend:

```
const login = async () => {
  const res = await axios.post('/api/auth/login', credentials);
  localStorage.setItem('token', res.data.token);
};
```

Attach token:

```
axios.get('/api/secure-notes', {
  headers: { Authorization: `Bearer ${token}` }
});
```

Absolutely! Let's level up with **trickier bugs** and **more realistic features** — just like what you might face in a real exam where you're given a half-working MERN app.

ADVANCED BUGS (and Their Fixes)

Bug 7: `useEffect` fetch runs twice

Cause: You're in React 18+ (StrictMode runs effects twice in dev)

```
useEffect(() => {
  fetchData();
}, []);
```

✅ **Fix (in dev only):** It's expected. To reduce confusion, disable StrictMode:

```
// index.js
<React.StrictMode>   ❌
  <App />
</React.StrictMode>
```

Or move logic inside:

```
useEffect(() => {
  let mounted = true;
  fetchData().then(() => {
    if (mounted) setNotes(...)
  });
  return () => mounted = false;
}, []);
```

Bug 8: React form submits and reloads the page

```
<form onSubmit={handleSubmit}>
  ...
</form>
```

```
const handleSubmit = () => {  
  // ❌ Missing e.preventDefault()  
}
```

✅ **Fix:**

```
const handleSubmit = (e) => {  
  e.preventDefault(); // ✅ Stop full reload  
  ...  
};
```

🐛 **Bug 9: Mongoose not saving updated data**

```
await Note.findByIdAndUpdate(req.params.id, req.body);  
res.json(note); // ❌ note is undefined
```

✅ **Fix:**

```
const updated = await Note.findByIdAndUpdate(req.params.id, req.body,  
{ new: true });  
res.json(updated); // ✅ Send updated note
```

🐛 **Bug 10: Mongo filter not working**

```
const query = { title: req.query.title };
```

✅ **Fix with regex:**

```
const query = req.query.title  
  ? { title: { $regex: req.query.title, $options: 'i' } }  
  : {};
```

🐛 **Bug 11: Axios call returns 401 unauthorized**

✅ **Check:**

- Token stored in localStorage?
- Token sent in headers?
- Backend route has authMiddleware?

✅ **Fix:**

```
axios.get('/api/notes', {  
  headers: { Authorization: `Bearer ${localStorage.getItem('token')}` }  
});
```

🌟 ADVANCED FEATURES (with Code Examples)

🌟 Feature 6: Toggle "Show Completed Notes Only"

Frontend:

```
const [showCompleted, setShowCompleted] = useState(false);
const filtered = notes.filter(note => !showCompleted || note.completed);

<button onClick={() => setShowCompleted(!showCompleted)}>
  {showCompleted ? 'Show All' : 'Show Completed'}
</button>
```

Backend Schema:

```
completed: { type: Boolean, default: false }
```

🌟 Feature 7: Add Checkbox to Mark Note as Completed

Frontend:

```
<input type="checkbox" checked={note.completed}
  onChange={() => toggleComplete(note._id)} />

const toggleComplete = async (id) => {
  const note = notes.find(n => n._id === id);
  const res = await axios.put(`/api/notes/${id}`, {
    ...note, completed: !note.completed
  });
  setNotes(notes.map(n => n._id === id ? res.data : n));
};
```

🌟 Feature 8: Show how many notes exist per tag

Backend:

```
app.get('/api/stats/tags', async (req, res) => {
  const stats = await Note.aggregate([
    { $group: { _id: "$tag", count: { $sum: 1 } } }
  ]);
  res.json(stats);
});
```

Frontend:

```
useEffect(() => {
  axios.get('/api/stats/tags').then(res => setStats(res.data));
}, []);
```

```
{stats.map(s => (  
  <li key={s._id}>{s._id}: {s.count}</li>  
))}
```

🌟 Feature 9: Pagination

Backend:

```
app.get('/api/notes', async (req, res) => {  
  const page = parseInt(req.query.page) || 1;  
  const limit = 5;  
  const notes = await Note.find()  
    .skip((page - 1) * limit)  
    .limit(limit);  
  res.json(notes);  
});
```

Frontend:

```
const [page, setPage] = useState(1);  
useEffect(() => {  
  axios.get(`/api/notes?page=${page}`).then(res => setNotes(res.data));  
}, [page]);
```

🌟 Feature 10: Sort by Title or Date

Backend:

```
app.get('/api/notes', async (req, res) => {  
  const sort = req.query.sort || 'createdAt';  
  const notes = await Note.find().sort({ [sort]: 1 });  
  res.json(notes);  
});
```

Frontend:

```
<select onChange={e => setSort(e.target.value)}>  
  <option value="title">Sort by Title</option>  
  <option value="createdAt">Sort by Date</option>  
</select>
```

🐛 Love that energy! Here's **another power-packed set of tricky bugs and real-world features** — exactly like what you'd see in **Web Engineering project exams**.

🐞 TRICKIER BUGS + FIXES

Bug 12: Cannot read properties of undefined (reading 'title')

Code:

```
<input value={form.title} onChange={handleChange} />
// But state was:
const [form, setForm] = useState();
```

✅ Fix: Initialize state

```
const [form, setForm] = useState({ title: '', content: '' });
```

Bug 13: React checkbox doesn't work

Code:

```
<input type="checkbox" checked={form.done} onChange={handleChange} />
```

✅ Fix: handle checkbox value separately

```
const handleChange = (e) => {
  const { name, value, type, checked } = e.target;
  setForm({ ...form, [name]: type === 'checkbox' ? checked : value });
};
```

Bug 14: React key warning in .map()

Code:

```
{notes.map((note) => (
  <li>{note.title}</li> // ❌ Missing key
))}
```

✅ Fix:

```
{notes.map((note) => (
  <li key={note._id}>{note.title}</li>
))}
```

Bug 15: fetch returns old data after submitting a new note

Likely cause:

- Fetch happens before the new note is added to MongoDB

✅ Fix: Wait for POST to complete, then fetch again or update UI manually

```
const handleSubmit = async () => {
```

```
    await axios.post('/api/notes', form);
    fetchNotes(); // re-fetch updated list
  };

```

🐛 Bug 16: MongoDB unique constraint not working

Schema:

```
username: String // ❌
```

✅ Fix:

```
username: { type: String, required: true, unique: true }
```

Also ensure index:

```
db.users.createIndex({ username: 1 }, { unique: true })
```

💡 ADVANCED FEATURES + CODE

💡 Feature 11: Dark/Light Mode Toggle in React

```
const [dark, setDark] = useState(false);

<button onClick={() => setDark(!dark)}>
  {dark ? 'Light Mode' : 'Dark Mode'}
</button>

<div className={dark ? 'dark-theme' : 'light-theme'}>
  ...
</div>
```

💡 Optional: Save preference to localStorage

💡 Feature 12: Auto-Save Notes While Typing

```
useEffect(() => {
  const timeout = setTimeout(() => {
    if (form.title) axios.put(`/api/notes/${form._id}`, form);
  }, 1000);

  return () => clearTimeout(timeout);
}, [form]);
```

💡 Feature 13: Protect Routes (React)

```
const token = localStorage.getItem('token');  
if (!token) return <Navigate to="/login" />;
```

✅ Use inside route component to prevent access

💡 Feature 14: Show “No Notes Found” If List is Empty

```
{notes.length === 0  
  ? <p>No notes available</p>  
  : notes.map(note => <NoteCard note={note} />) }
```

💡 Feature 15: Show Relative Dates (e.g., "2 hours ago")

```
npm install dayjs  
import dayjs from 'dayjs';  
import relativeTime from 'dayjs/plugin/relativeTime';  
dayjs.extend(relativeTime);  
  
<p>{dayjs(note.createdAt).fromNow()}</p>
```

💡 Feature 16: Sort Notes by Newest First

Backend:

```
const notes = await Note.find().sort({ createdAt: -1 });
```

Ensure your schema has:

```
timestamps: true
```

💡 Feature 17: Add Note Priority (Low/Medium/High)

Schema:

```
priority: { type: String, enum: ['Low', 'Medium', 'High'], default:  
  'Medium' }
```

Frontend:

```
<select name="priority" value={form.priority} onChange={handleChange}>  
  <option>Low</option>  
  <option>Medium</option>  
  <option>High</option>  
</select>
```

Style by color:

```
<span className={`tag
${note.priority.toLowerCase()}`}>{note.priority}</span>
```

🌟 Feature 18: Export Notes to CSV

```
const exportCSV = () => {
  const rows = notes.map(n => `${n.title},${n.content},${n.tag}`);
  const blob = new Blob(['Title,Content,Tag\n${rows.join('\n')}'], { type:
'text/csv' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = 'notes.csv';
  a.click();
};
```

You got it! Here's a **combo pack** of:

✅ 10 Real-World Web Features to Implement (Not Just Notes App!)

Each one is exam-style, realistic, and based on MERN logic you already know.

🔧 FEATURE PROMPTS (Project-Agnostic)

📦 Implement a Like System

Each item (e.g., blog, product, tutor) should have a “Like” button. Users can toggle it.

- Schema: likes: { type: Number, default: 0 }
 - Route: PUT /api/item/:id/like
 - Frontend: toggle button that increments/decrements and updates UI
-

📦 Allow Users to Upload Profile Pictures

Add profile image upload to user dashboard.

- Use `multer` for file upload
 - Store image path in MongoDB
 - Show preview using `img src={user.profilePic}`
-

❑ Implement Role-Based Routing

Some routes only work for admin users.

- Backend middleware:

```
if (req.user.role !== 'admin') return res.status(403).json({ error: 'Forbidden' });
```

- Protect routes like `GET /api/users`
-

❑ Add Reset Password Feature

User can enter email, get a link to reset their password.

- Send token (mock or console log)
 - Store token with expiry
 - New route: `POST /api/reset-password`
-

❑ Add Favorite Items to a Wishlist

Toggle heart icon on items to add/remove from wishlist.

- Schema: `wishlist: [itemId]`
 - Route: `PUT /api/users/:id/wishlist`
 - Frontend: 🇨🇳 / 💖 icon changes based on inclusion
-

❑ Filter Products by Price Range + Category

Build a dynamic filter sidebar.

- Backend: `GET /api/products?min=100&max=500&category=laptop`
- Mongo filter:

```
Product.find({
  price: { $gte: req.query.min, $lte: req.query.max },
  category: req.query.category
```

```
}}
```

📌 Add Comment Section with Timestamp

Each post has comments that show relative time.

- `comments: [{ userId, text, createdAt }]`
 - Use `dayjs().fromNow()`
-

📌 Implement Infinite Scroll

Auto-load more data when user scrolls down.

- `GET /api/items?skip=20&limit=10`
- In React, detect scroll near bottom:

```
window.onscroll = () => { if (window.innerHeight + window.scrollY >= document.body.offsetHeight) loadMore(); };
```

📌 Add Notification System

When someone posts, others get notified.

- Notifications schema
 - New route: `POST /api/notifications`
 - Show badge in navbar: 🔔 3
-

10 📌 Display Data in Calendar View

Visualize session/bookings per day (like tutor sessions).

- Use `react-calendar` or `fullcalendar.io`
 - Backend: `GET /api/sessions`
 - Frontend: Map dates to events
-

🐛 BUG SPOTTING CHALLENGES

Can you spot and fix these bugs?

Bug A: React input doesn't update

```
<input type="text" value={form.title} onChange={handleChange} />
const handleChange = () => {
  setForm({ title: e.target.value });
}
```

✗ What's wrong?

✓ **Fix:**

```
const handleChange = (e) => {
  setForm({ ...form, [e.target.name]: e.target.value });
};
```

Bug B: Express route not responding

```
app.get('/api/items', (req, res) => {
  res.send('Items loaded');
})
```

✗ What's wrong?

✓ **Fix:** Missing `)` at the end.

Bug C: Axios not sending Authorization header

```
axios.get('/api/secure-data');
```

✗ Fix: No token sent

✓ **Fix:**

```
axios.get('/api/secure-data', {
  headers: { Authorization: `Bearer ${localStorage.getItem('token')}` }
});
```

Bug D: Cannot read property 'map' of undefined

```
notes.map(note => <li>{note.title}</li>);
```

✗ Fix: `notes` is undefined initially

✓ **Fix:**

```
{notes?.map(note => ...) }
```

or

```
const [notes, setNotes] = useState([]);
```

Bug E: useEffect infinite loop

```
useEffect(() => {  
  fetchNotes();  
}, [notes]); // ❌ triggers on every notes update
```

Fix:

```
useEffect(() => {  
  fetchNotes();  
}, []); // ✅ only on mount
```

 Let's go! Here are the **complete code solutions** for the **10 real-world feature prompts** I gave earlier. These aren't just notes-app tricks — they'll help you handle **any MERN exam scenario** with confidence.

FULL CODE SOLUTIONS FOR 10 REAL FEATURES

📌 Like System

Schema (e.g. Post.js)

```
likes: { type: Number, default: 0 }
```

Backend route:

```
app.put('/api/posts/:id/like', async (req, res) => {  
  const post = await Post.findById(req.params.id);  
  post.likes += 1;  
  await post.save();  
  res.json(post);  
});
```

Frontend (React):

```
<button onClick={() => likePost(post._id)}> {post.likes}</button>
```

```
const likePost = async (id) => {  
  const res = await axios.put(`/api/posts/${id}/like`);  
  setPosts(posts.map(p => p._id === id ? res.data : p));  
}
```



```
};
```

✅ 11 Profile Picture Upload (Multer)

Backend setup:

```
npm install multer
```

Backend:

```
const multer = require('multer');
const storage = multer.diskStorage({
  destination: 'uploads/',
  filename: (req, file, cb) => cb(null, Date.now() + '-' +
file.originalname)
});
const upload = multer({ storage });

app.post('/api/profile-pic', upload.single('image'), async (req, res) => {
  const user = await User.findById(req.body.userId);
  user.profilePic = req.file.path;
  await user.save();
  res.json(user);
});
```

Frontend:

```
<input type="file" onChange={e => setFile(e.target.files[0])} />
<button onClick={uploadPic}>Upload</button>

const uploadPic = async () => {
  const formData = new FormData();
  formData.append('image', file);
  formData.append('userId', user._id);
  const res = await axios.post('/api/profile-pic', formData);
  setUser(res.data);
};
```

✅ 12 Role-Based Route Protection

Backend middleware:

```
const requireAdmin = (req, res, next) => {
  if (req.user.role !== 'admin') return res.status(403).json({ error:
'Forbidden' });
  next();
};

app.get('/api/admin-data', authMiddleware, requireAdmin, (req, res) => {
  res.send('Secret admin stuff');
});
```

✅ Reset Password Flow (Simulated)

Schema:

```
resetToken: String,  
resetExpires: Date
```

Backend:

```
app.post('/api/request-reset', async (req, res) => {  
  const token = Math.random().toString(36).substring(2);  
  await User.findOneAndUpdate({ email: req.body.email }, {  
    resetToken: token,  
    resetExpires: Date.now() + 3600000  
  });  
  console.log(`Use this token to reset: ${token}`);  
  res.send('Token sent');  
});  
  
app.post('/api/reset-password', async (req, res) => {  
  const user = await User.findOne({  
    resetToken: req.body.token,  
    resetExpires: { $gt: Date.now() }  
  });  
  user.password = await bcrypt.hash(req.body.newPassword, 10);  
  user.resetToken = undefined;  
  user.resetExpires = undefined;  
  await user.save();  
  res.send('Password updated');  
});
```

✅ Wishlist Toggle

Schema:

```
wishlist: [String] // array of item IDs
```

Backend route:

```
app.put('/api/users/:id/wishlist', async (req, res) => {  
  const user = await User.findById(req.params.id);  
  const itemId = req.body.itemId;  
  const index = user.wishlist.indexOf(itemId);  
  if (index > -1) {  
    user.wishlist.splice(index, 1); // remove  
  } else {  
    user.wishlist.push(itemId); // add  
  }  
  await user.save();  
  res.json(user);  
});
```

Frontend:

```
<button onClick={() => toggleWishlist(item._id)}>
```

```
    {user.wishlist.includes(item._id) ? '❤️' : '💔'}
  </button>
```

✅ 6 Filter Products by Price & Category

Backend:

```
app.get('/api/products', async (req, res) => {
  const { min = 0, max = 9999, category } = req.query;
  const query = {
    price: { $gte: min, $lte: max }
  };
  if (category) query.category = category;
  const products = await Product.find(query);
  res.json(products);
});
```

Frontend:

```
axios.get(`/api/products?min=100&max=500&category=laptop`);
```

✅ 7 Comment Section with Timestamps

Schema:

```
comments: [{ user: String, text: String, createdAt: { type: Date, default:
Date.now } }]
```

Backend route:

```
app.post('/api/posts/:id/comment', async (req, res) => {
  const post = await Post.findById(req.params.id);
  post.comments.push({ user: req.body.user, text: req.body.text });
  await post.save();
  res.json(post);
});
```

Frontend:

```
<p>{dayjs(comment.createdAt).fromNow()}</p>
```

✅ 8 Infinite Scroll

Backend:

```
app.get('/api/posts', async (req, res) => {
  const skip = parseInt(req.query.skip) || 0;
  const posts = await Post.find().skip(skip).limit(10);
  res.json(posts);
});
```

Frontend:

```
const loadMore = async () => {
  const res = await axios.get(`/api/posts?skip=${posts.length}`);
  setPosts([...posts, ...res.data]);
};
```

✅ 9 Notification System

Schema:

```
notifications: [{ text: String, read: Boolean, createdAt: Date }]
```

Backend:

```
app.post('/api/notifications', async (req, res) => {
  const user = await User.findById(req.body.userId);
  user.notifications.push({ text: req.body.text, read: false, createdAt:
new Date() });
  await user.save();
  res.json(user);
});
```

Frontend:

```
<p>🔔 {user.notifications.filter(n => !n.read).length}</p>
```

✅ 10 Calendar View

Use [react-calendar](#) or [FullCalendar](#).

Backend:

```
app.get('/api/sessions', async (req, res) => {
  const sessions = await Session.find();
  res.json(sessions.map(s => ({
    title: s.topic,
    date: s.date
  })));
});
```

Frontend:

```
import Calendar from 'react-calendar';

<Calendar
  tileContent={({ date }) => {
    const hasSession = sessions.some(s => s.date === formatDate(date));
    return hasSession ? <span>📅</span> : null;
  }}
/>
```

Great — let's break it down step-by-step and go through how a typical **MERN stack project** is structured and how each part works according to your exam topics.

1. Project Folder Structure (Workflow Overview)

```
my-mern-app/
├── backend/
│   ├── models/           # Mongoose schemas
│   ├── routes/           # Express routes (CRUD APIs)
│   ├── controllers/      # Functions handling logic
│   ├── views/            # EJS files if using templating
│   ├── app.js            # Express app setup
│   └── .env              # Environment variables
├── frontend/
│   ├── public/           # Static files
│   ├── src/
│   │   ├── components/  # React components
│   │   ├── pages/       # Page-level components
│   │   ├── App.js       # Main app component
│   │   └── index.js     # Entry point
│   └── package.json
├── .gitignore
├── package.json (root or separate for each)
└── README.md
```

2. Node.js & NPM (Backend)

- NPM:

- Manages dependencies.
- `npm init -y` initializes `package.json`
- Install packages: `npm install express mongoose cors dotenv`

- Node Modules:

- Folder where dependencies are stored.
- Avoid pushing to GitHub (`.gitignore` it).

- CommonJS vs ES6:

```
// CommonJS (default in Node.js)
const express = require('express');

// ES6 Modules (if "type": "module" in package.json)
import express from 'express';
```

3. Express (Backend)

- Express App (`app.js`)

```
const express = require('express');
const app = express();
const cors = require('cors');

app.use(cors());
app.use(express.json()); // middleware to parse JSON
app.use('/api/notes', require('./routes/noteRoutes'));

app.set('view engine', 'ejs'); // If using EJS

module.exports = app;
```

- Middleware:

- `express.json()` for parsing JSON
- `cors()` for allowing cross-origin requests

- CRUD APIs (`routes/noteRoutes.js`)

```
const express = require('express');
const router = express.Router();
const Note = require('../models/Note');

// CREATE
router.post('/', async (req, res) => {
  const newNote = new Note(req.body);
  await newNote.save();
  res.json(newNote);
});

// READ
router.get('/', async (req, res) => {
  const notes = await Note.find();
  res.json(notes);
});

// UPDATE
router.put('/:id', async (req, res) => {
  const updated = await Note.findByIdAndUpdate(req.params.id, req.body,
{ new: true });
  res.json(updated);
});

// DELETE
router.delete('/:id', async (req, res) => {
  await Note.findByIdAndDelete(req.params.id);
  res.sendStatus(204);
});

module.exports = router;
```

4. MongoDB with Mongoose

- Connect (backend/app.js)

```
const mongoose = require('mongoose');
mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
```

- Schema (models/Note.js)

```
const mongoose = require('mongoose');

const NoteSchema = new mongoose.Schema({
  title: String,
  content: String,
});

module.exports = mongoose.model('Note', NoteSchema);
```

- Mongo Queries

```
Note.find(); // read all
Note.findById(id); // read one
Note.create({ title, content }); // create
Note.findByIdAndUpdate(id, update); // update
Note.findByIdAndDelete(id); // delete
```

5. React (Frontend)

- JSX & Components (src/components/NoteCard.js)

```
function NoteCard({ note }) {
  return (
    <div>
      <h3>{note.title}</h3>
      <p>{note.content}</p>
    </div>
  );
}
export default NoteCard;
```

- State, Events, Conditional Rendering

```
import { useState } from 'react';

function NoteForm({ onAdd }) {
  const [title, setTitle] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    onAdd(title);
    setTitle('');
  };
}
```

```

    };

    return (
      <form onSubmit={handleSubmit}>
        <input value={title} onChange={(e) => setTitle(e.target.value)} />
        <button type="submit">Add</button>
      </form>
    );
  }
}

```

- Sharing State

Parent holds state:

```

const [notes, setNotes] = useState([]);
<NoteForm onAdd={(newTitle) => setNotes([...notes, { title: newTitle }])} />
/>

```

6. MERN Integration (Frontend to Backend)

- Fetch Notes (GET)

```

useEffect(() => {
  fetch('/api/notes')
    .then(res => res.json())
    .then(data => setNotes(data));
}, []);

```

- Add Note (POST)

```

fetch('/api/notes', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ title: "New", content: "Hello" }),
});

```

7. Authentication (Login / Sign Up)

- Auth Flow:

1. User submits login/signup form.
 2. Backend checks credentials.
 3. Backend returns JWT token.
 4. Frontend stores token (localStorage).
 5. Frontend uses token in headers for protected requests.
-

